

Context-Switch Adaptive TLB Prefetching

UNIST CSE Course Project Report, June 2025

Joanita Nakityo

Ulsan National Institute of Science and Technology
(UNIST)

Department of Computer Science and Engineering
Ulsan, South Korea

Hyeonseok Oh

Ulsan National Institute of Science and Technology
(UNIST)

Department of Computer Science and Engineering
Ulsan, South Korea

Abstract

Efficient virtual-to-physical address translation is essential for modern processors, and the Translation Lookaside Buffer (TLB) plays a central role in this process. However, context switches introduce cold TLB misses due to a mismatch between the incoming process's memory access patterns and the current TLB contents. To address this, we introduce a context-aware TLB prefetching mechanism that maintains per-process prefetcher state across context switches. Our design associates each process with a dedicated prefetch queue and history, which are saved and restored alongside the process control block (PCB), enabling warm-start behavior after a switch. We extend the ChampSim simulator to support multi-process workloads with frequent context switches and evaluate our design using key metrics including TLB hit ratio, prefetch queue hit ratio, and instructions per cycle (IPC). Our results show that preserving per-process prefetcher state significantly reduces cold misses and improves IPC during the recovery phase after a context switch..

CCS Concepts: • Computer systems organization → Parallel architectures.

Keywords: TLB Prefetching, Context Switching, Virtual Memory Management

1 Introduction

In modern computer systems, the Translation Lookaside Buffer (TLB) plays a vital role in accelerating virtual-to-physical address translation. When a memory access occurs, the processor first consults the TLB for a cached translation. If the required entry is not found—a TLB miss—the system must initiate a page table walk, incurring substantial latency and additional memory references.

A common source of TLB misses is a context switch, which occurs when the operating system switches execution from one process to another. Although many modern architectures support Address Space Identifiers (ASIDs) [1] to retain TLB entries across context switches and avoid full TLB flushes, cold misses still persist. This is because the incoming process typically exhibits a different memory access pattern, and the TLB lacks useful entries to serve it efficiently.

To mitigate such misses, prior work has introduced TLB prefetchers that speculate on future address translations

and pre-load them into the TLB [4, 13, 16, 20]. While effective in certain scenarios, most existing prefetchers adopt a one-size-fits-all approach, which often fails after a context switch due to the sudden shift in memory access behavior. As highlighted in prior work such as the Agile TLB Prefetcher (ATP) [23], no single prefetching strategy consistently outperforms others across all applications. ATP adapts the choice of prefetcher based on runtime behavior and utilizes Sampling-Based Free TLB Prefetching (SBFP) to improve efficiency by leveraging page table locality.

However, existing techniques—including ATP and SBFP—do not address the loss of prefetcher state across context switches, which resets learning and results in repeated cold starts, even when ASIDs are in use.

Our Contribution In this paper, we propose a context-aware TLB prefetching mechanism that preserves per-process prefetcher state across context switches. Our key insight is that the predictive behavior of prefetchers can be made process-specific. We therefore associate each process with its own Prefetch Queue (PQ) and prefetch history, both of which are saved and restored as part of the process control block (PCB) during a context switch. This allows each process to resume with a “warm” prefetcher state, significantly reducing cold TLB misses.

To evaluate our approach, we extend the ChampSim simulator, originally designed for single-context execution, to support multi-process workloads and frequent context switching. Our modified simulator provides a robust platform for analyzing memory behavior under realistic multi-programming conditions.

This paper makes the following contributions:

- We propose an adaptive, context-aware TLB prefetching framework that maintains per-process prefetch queues and histories, enabling processes to retain their learned memory access patterns across context switches.
- We extend the ChampSim simulator to model context switching and multi-process execution, enabling detailed evaluation of our technique under realistic workloads.
- We demonstrate that our approach significantly reduces cold TLB misses and improves performance

compared to traditional prefetchers that reset state upon every context switch.

2 Background and Motivation

This section provides an overview of virtual memory systems, TLB behavior, context switching, and TLB prefetching strategies, which are essential to understanding the challenges addressed in this work.

2.1 Virtual Memory and Address Translation

Modern operating systems use virtual memory to provide each process with the illusion of a large, private, and contiguous address space. To access data in physical memory, the processor must translate virtual addresses (used by programs) into physical addresses (used by the hardware). This translation is performed using a page table, which maps virtual pages to physical frames [3, 5, 9, 10]. Accessing the page table directly for every memory reference is inefficient due to the overhead of multiple memory lookups. To mitigate this, processors rely on fast address translation caches called Translation Lookaside Buffers (TLBs).

2.2 Translation Lookaside Buffer (TLB)

A Translation Lookaside Buffer (TLB) is a small, hardware-managed cache that stores recently used virtual-to-physical address mappings. When the processor accesses a virtual address, it first checks the TLB:

- If the mapping is found (a TLB hit), the address translation completes quickly.
- If the mapping is not found (a TLB miss), the processor must perform a page walk, which traverses the page table hierarchy to resolve the address. This process is slower and introduces a performance penalty.

On a TLB miss, as illustrated in Figure 1 similar to prior work [14], the CPU performs a page walk to translate the virtual address. In the x86-64 architecture, this involves navigating a four-level radix tree page table. The base address of the top-level page table (PML4) is stored in the CR3 register.

The virtual address is divided into five parts: four 9-bit indices that correspond to each level of the page table hierarchy—namely, the Page Map Level 4 (PML4), Page Directory Pointer Table (PDPT), Page Directory (PD), and Page Table (PT)—followed by a 12-bit page offset. The CPU first uses the CR3 register to locate the PML4 table and uses the PML4 index to read the base address of the PDPT. It then uses the PDPT index to find the PD, and the PD index to reach the PT. Finally, the PT index is used to access the physical page frame number stored in the leaf-level entry. This physical frame number is combined with the page offset to compute the full physical address. Once the translation is complete, the mapping is cached in the TLB.

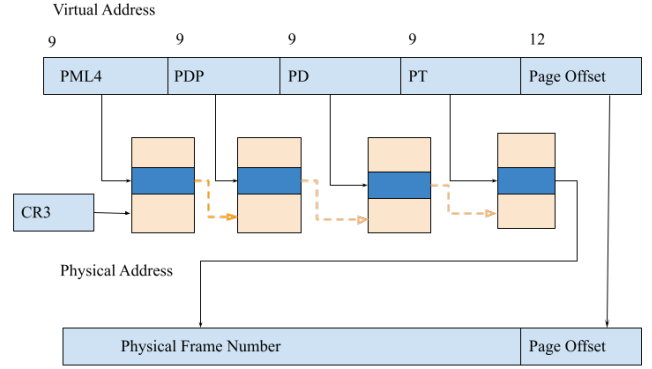


Figure 1. Page Walk in x86-64 Architecture Showing the Translation from Virtual Address to Physical Address Through PML4, PDPT, PD, and PT Levels

2.3 Context Switching and Its Impact on the TLB

A context switch occurs when the operating system (OS) suspends one process and resumes another on the same core. During this switch, the TLB must invalidate entries belonging to the previous process because its virtual-to-physical mappings are no longer relevant. Some architectures assign each process an Address Space Identifier (ASID) that tags TLB entries, allowing the OS to retain certain entries for different processes. However, even with ASIDs, a newly scheduled process still faces a “cold-start” penalty: the TLB is effectively empty for its working set, which causes a surge in TLB misses and subsequent page walks until the process’s hot pages are reloaded.

To hide this latency, modern processors often implement TLB prefetchers, which predict future page table entries and insert them into the TLB before references actually occur. In theory, prefetchers can substantially reduce page-walk overhead by proactively populating the TLB. In practice, however, most prefetchers are shared across all processes and rely on static prediction heuristics. For instance, a streaming-based prefetcher might excel at sequential memory scans but perform poorly on random or pointer-chasing workloads. Because these prefetchers do not adapt per process and reset their internal state upon each context switch, they must “re-learn” a new process’s access patterns from scratch, increasing the cold-start penalty.

Figure 2 illustrates this sequence in detail. Initially, Process P0 executes on a CPU core, and its TLB contains P0’s current page mappings. The TLB’s prefetcher is tuned to P0’s access stream, reducing P0’s miss rate. When an interrupt or system call forces the OS to switch to Process P1, the OS flushes—or logically invalidates—the TLB entries for P0. P1 begins execution with no valid TLB entries, causing a high miss rate and multiple page walks until the hot pages are loaded. Meanwhile, the prefetcher has lost any history of P0’s streaming pattern and must gather new hints about P1’s

behavior. Because of this, P1 experiences many TLB misses and poor prefetcher performance until its memory access pattern becomes clear. Meanwhile, P0 just remains idle until the OS schedules it again.

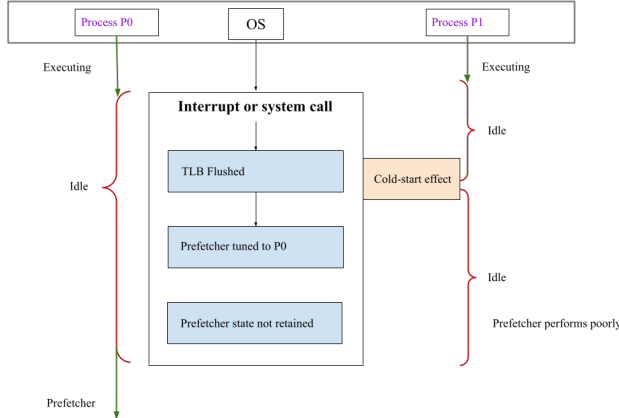


Figure 2. Illustration of the context switch and its impact on the TLB and prefetcher.

This example shows that context switches impose two overlapping overheads: first, flushing (or remapping) the TLB, and second, resetting the prefetcher’s predictive state. Even when ASIDs reduce the frequency of a full flush, they cannot avoid the cold-start effect on each newly scheduled process.

2.4 Motivation

Translation Lookaside Buffers (TLBs) are critical for hiding the latency of virtual-to-physical address translation. However, TLB effectiveness degrades significantly after a context switch, especially in systems running multiple applications concurrently. Despite support for Address Space Identifiers (ASIDs), many context switches still result in TLB cold misses that lead to costly page walks. Existing TLB prefetchers attempt to mitigate these misses but are typically global and oblivious to process-specific access patterns. This limits their ability to provide accurate prefetches after a context switch, as the memory behavior of one process may not resemble that of another. Additionally, most TLB prefetchers use static heuristics, which fail to adapt to the diverse and dynamic memory access patterns found in modern workloads. To address this, there is a need for a more adaptive and process-aware TLB prefetching mechanism—one that can retain useful state across context switches and tailor its predictions to the memory behavior of each process individually. Such a mechanism would reduce cold-start overheads and improve memory system performance in multi-program environments. We build on the insights obtained from the previous work [15, 23]

3 Implementation

Our implementation consists of two main components: a three-stage TLB translation and prefetching workflow, and a context-switching mechanism for simulating multiprogrammed execution.

3.1 Translation and Prefetching Workflow

To reduce TLB miss latency, we implemented a prefetching mechanism that operates in three stages:

1. **SPQ Lookup:** On a TLB miss, the virtual address is first looked up in a Shared Page Queue (SPQ), which caches recently resolved virtual-to-physical (v2p) translations. If a match is found, translation completes immediately, and the mapping is inserted into the TLB.
2. **FPQ Matching:** If the SPQ does not contain the translation, each TLB prefetcher generates predicted virtual addresses, which are inserted into its corresponding Future Prefetch Queue (FPQ). The missed virtual address is then compared against the entries in all FPQs. If a match is found, the corresponding prefetcher’s counter is incremented or decremented based on prediction accuracy.
3. **Prefetcher Selection and Execution:** After all FPQs are checked, the counters are used to guide a decision tree that selects the most effective prefetcher. The selected prefetcher then generates its own predicted virtual addresses, which are issued as prefetches. These translations are resolved via simulated page walks and inserted into the SPQ only. TLB insertion occurs only upon actual demand misses.

This adaptive workflow enables the system to dynamically identify and utilize the most suitable prefetcher based on recent prediction performance.

3.2 Context Switching Support

To simulate a realistic multiprogrammed environment, we implemented periodic context switching by alternating between multiple application traces. The following actions occur at each context switch:

- **TLB Flush:** Both instruction and data TLBs are flushed to model address space isolation.
- **State Preservation:** The SPQ, all FPQs, and associated prediction counters are saved and restored for each process. This preserves prefetching state across context switches, reflecting operating system behavior where prediction context is often retained.
- **Trace Management:** The core switches to the next trace in a predefined sequence after a fixed number of cycles. Execution resumes from where the process left off, along with its corresponding prediction state.

This setup allows us to examine how well TLB prefetchers retain accuracy across context switches and how history-based predictions perform under interleaved task execution.

Table 1. Hardware Configuration

Component	Configuration
L1 ITLB	64-entry, 4-way, 1-cycle, 4-entry MSHR
L1 DTLB	64-entry, 4-way, 1-cycle, 4-entry MSHR
L2 TLB	1536-entry, 12-way, 8-cycle, 4-entry MSHR
Prefetch Queue	64-entry, fully associative, 2-cycle
Sampler	64-entry, fully associative, 2-cycle
L1 I-Cache	32KB, 8-way, 1-cycle, 8-entry MSHR
L1 D-Cache	32KB, 8-way, 4-cycle, 8-entry MSHR
L2 Cache	256KB, 8-way, 8-cycle, 16-entry MSHR
LLC	2MB, 16-way, 20-cycle, 32-entry MSHR
DRAM	4GB, tRP = tRCD = tCAS = 11

4 Experimental Setup

4.1 workloads

We evaluate our proposed TLB prefetching mechanism using two memory-intensive workloads: MILC [6] and LBM. MILC is a quantum chromodynamics (QCD) simulation with complex memory access patterns, while LBM is a computational fluid dynamics application that exhibits regular spatial locality. These workloads were selected due to their high TLB miss intensities and diverse memory behaviors. All traces are collected using the SimPoint methodology, with each workload executing about 1 billion instructions for measurement.

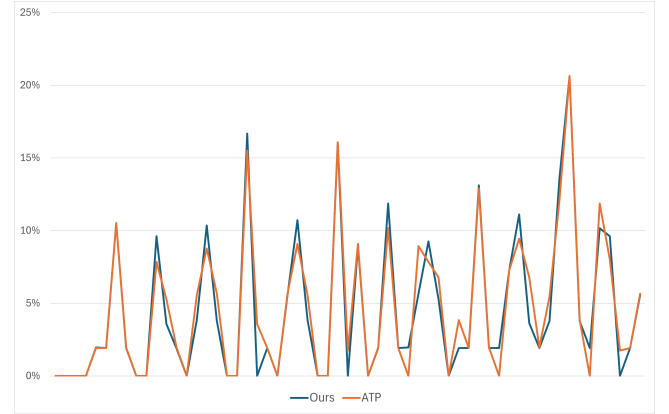
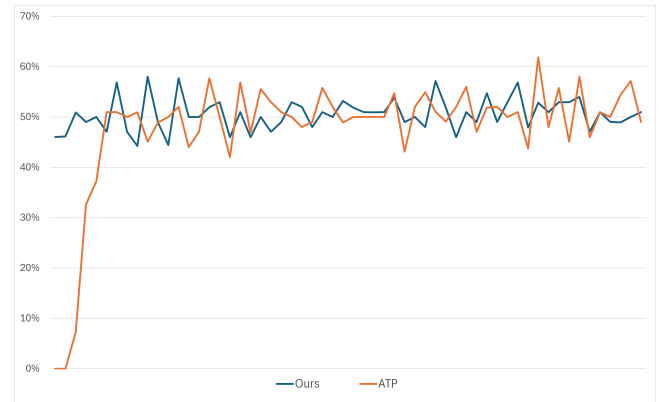
To simulate a multiprogrammed environment, we introduce periodic context switches between MILC and LBM at fixed instruction intervals (e.g., every 600 thousand cycles). During each switch, the simulator saves the current process’s TLB prefetcher state and restores the corresponding state for the next process. This mechanism enables the evaluation of process-aware prefetching techniques that retain useful information across context switches.

4.2 Metrics

We evaluate our approach using three primary metrics: TLB hit rate, which captures the frequency of TLB misses; prefetch queue (PQ) hit ratio, which measures the proportion of memory accesses satisfied by the TLB prefetcher; and instruction throughput (IPC), representing overall processor performance.

4.3 Simulation Infrastructure

We use ChampSim [18], a detailed simulator that models a 4-wide out-of-order processor by default. ChampSim assumes a single-threaded execution model. To simulate multi-process behavior, we extended it to support context switching and per-process prefetch queues hence having the hardware configuration shown in Table 1

**Figure 3.** TLB Hit Ratio comparison after a context switch.**Figure 4.** PQ Hit Ratio comparison after a context switch.

5 Evaluation

To evaluate the effectiveness of our technique, we compared its performance against ATP. Specifically, we observed the changes in Instructions Per Cycle (IPC), PQ Hit Ratio, and TLB Hit Ratio at intervals of 1000 cycles following a context switch.

5.1 TLB Hit Ratio

The TLB Hit Ratio represents the probability that a page table entry is found in the Shared TLB (STLB) upon a translation request. Since our design transfers entries that hit in the SPQ to the TLB, we anticipated an increase in the TLB Hit Ratio.

As shown in Figure 3, the overall TLB Hit Ratio did not exhibit a significant change. However, at certain peak points approximately 3000 cycles after the context switch, our technique demonstrated a higher TLB Hit Ratio compared to ATP.

This observation suggests that entries that previously hit in the SPQ were subsequently loaded into the TLB and eventually contributed to TLB hits after a short delay.

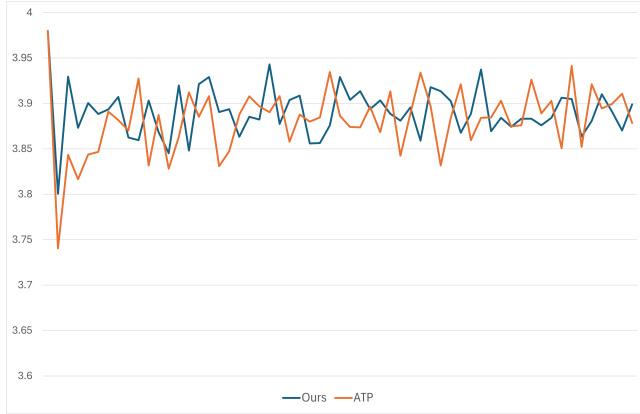


Figure 5. IPC comparison after a context switch .

5.2 PQ Hit Ratio

The PQ Hit Ratio denotes the probability that a virtual address causing a TLB miss results in a hit in the SPQ. Unlike ATP, which flushes the SPQ upon a context switch, our approach preserves the SPQ by saving and restoring it. Consequently, we expected a more stable PQ Hit Ratio following context switches.

As illustrated in Figure 4, ATP failed to perform effective prefetching for approximately 3000 cycles after a context switch. In contrast, our approach maintained a consistent PQ Hit Ratio throughout this period.

5.3 Instructions Per Cycle (IPC)

IPC refers to the number of instructions executed per cycle. Given that our technique remains effective for approximately 3000 cycles following a context switch—unlike ATP—we expected it to improve IPC during that interval.

As depicted in Figure 5, our approach achieved approximately 0.05 additional instructions per cycle compared to ATP during the first 3000 cycles.

Furthermore, we observed that under ATP, the same instruction execution pattern appeared approximately 3000 cycles later, indicating that our technique advances the execution of such patterns.

6 Related Work

Virtual Memory: Virtual memory abstracts the physical memory by giving each process its own address space. The translation between virtual and physical addresses is typically handled via multi-level page tables. [3, 5, 9, 10]. Hashed page tables [11, 21, 24] have also been explored to reduce page walk latency, offering alternatives to traditional radix trees.

TLB Prefetchers: Many researchers have proposed TLB prefetchers to reduce the penalty of TLB misses. [4, 13, 16, 20] designed prefetchers that either share TLB entries across cores or exploit spatial locality in virtual memory. However,

these techniques often rely on fixed strategies or global behaviors, lacking the adaptability to individual process memory patterns—especially after context switches. More recent designs like SBFP exploit virtual contiguity for adaptive prefetching but still leave room for improvement in process-specific prediction after context switches.

Large TLB Designs: Prior works [2, 7, 8, 12, 15, 17, 19, 22] propose large, software-managed TLBs to reduce costly page walks. These designs complement hardware-based prefetching by mitigating the capacity limitations of traditional hardware TLBs, especially in workloads with large memory footprints or frequent reuse of virtual addresses.

7 Conclusion

This paper presents Context Switch Adaptive TLB Prefetching, a novel approach to cold misses that occur during the context switch. By preserving per-process prefetcher state across context switches, each process is allowed to resume with a “warm” prefetcher state. Our evaluation shows that preserving per-process prefetcher state reduces cold misses and improves IPC during the recovery phase after a context switch.

References

- [1] ARM Ltd. 2004. *ARM1136JF-S and ARM1136J-S Technical Reference Manual*. http://infocenter.arm.com/help/topic/com.arm.doc.ddi0211k/ddi0211k_arm1136_r1p5_trm.pdf.
- [2] Kavita Bala, M. Frans Kaashoek, and William E. Weihl. 1994. Software Prefetching and Caching for Translation Lookaside Buffers. In *Proceedings of the 1st USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [3] Abhishek Bhattacharjee. 2018. Breaking the Address Translation Wall By Accelerating Memory Replays. *IEEE Micro* (2018).
- [4] Abhishek Bhattacharjee and Margaret Martonosi. 2010. Inter-core Cooperative TLB for Chip Multiprocessors. In *Proceedings of the 15th Edition of ASPLOS on Architectural Support for Programming Languages and Operating Systems*. ACM, 359–370. doi:10.1145/1736020.1736063
- [5] Richard W. Carr and John L. Hennessy. 1981. WSCLOCK—a simple and effective algorithm for virtual memory management. In *Proceedings of the Eighth ACM Symposium on Operating Systems Principles* (Pacific Grove, California, USA) (SOSP '81). Association for Computing Machinery, New York, NY, USA, 87–95. doi:10.1145/800216.806596
- [6] National Energy Research Scientific Computing Center. [n.d.]. MILC Workload Description. https://www.olcf.ornl.gov/wp-content/uploads/OLCF-6_MILC_description.pdf.
- [7] Xiaotao Chang, Hubertus Franke, Yi Ge, Tao Liu, Kun Wang, Jimi Xenidis, Fei Chen, and Yu Zhang. 2013. Improving Virtualization in the Presence of Software-Managed Translation Lookaside Buffers. In *Proceedings of the 40th Annual International Symposium on Computer Architecture (ISCA)*.
- [8] D. R. Cheriton, G. A. Slavenburg, and P. D. Boyle. 1986. Software-Controlled Caches in the VMP Multiprocessor. In *Proceedings of the 13th Annual International Symposium on Computer Architecture (ISCA)*.
- [9] D.R. Engler, S.K. Gupta, and M.F. Kaashoek. 1995. AVM: application-level virtual memory. In *Proceedings 5th Workshop on Hot Topics in Operating Systems (HotOS-V)*. 72–77. doi:10.1109/HOTOS.1995.513458
- [10] Steven M Hand. 1999. Self-Paging in the Nemesis Operating System. In *Proceedings of OSDI*.

- [11] Jerry Huck and Jim Hays. 1993. Architectural support for translation table management in large address space machines. *SIGARCH Comput. Archit. News* 21, 2 (May 1993), 39–50. doi:10.1145/173682.165128
- [12] Bruce L. Jacob and Trevor N. Mudge. 1998. A Look at Several Memory Management Units, TLB-Refill Mechanisms, and Page Table Organizations. In *Proceedings of the 9th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [13] G. B. Kandiraju and A. Sivasubramaniam. 2002. Going the Distance for TLB Prefetching: An Application-driven Study. In *Proceedings of the 29th Annual International Symposium on Computer Architecture*. IEEE, 195–206.
- [14] Konstantinos Kanellopoulos, Hong Chul Nam, Nisa Bostanci, Rahul Bera, Mohammad Sadrosadati, Rakesh Kumar, Davide Basilio Bartolini, and Onur Mutlu. 2023. Victima: Drastically Increasing Address Translation Reach by Leveraging Underutilized Cache Resources. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (Toronto, ON, Canada) (MICRO '23)*. Association for Computing Machinery, New York, NY, USA, 1178–1195. doi:10.1145/3613424.3614276
- [15] Yashwant Marathe, Nagendra Gulur, Jee Ho Ryoo, Shuang Song, and Lizy K. John. 2017. CSALT: Context Switch Aware Large TLB. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*.
- [16] Aleksandar Margaritov, Dmitrii Ustiugov, Edouard Bugnion, and Boris Grot. 2019. Prefetched Address Translation. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1024–1036. doi:10.1145/3352460.3358267
- [17] David Nagle, Richard Uhlig, Tim Stanley, Stuart Sechrest, Trevor Mudge, and Richard Brown. 1993. Design tradeoffs for software-managed TLBs. *SIGARCH Comput. Archit. News* 21, 2 (May 1993), 27–38. doi:10.1145/173682.165127
- [18] Hyeonseok Oh. 2025. Modified ChampSim Simulator with ATP and Context Switching Support. <https://github.com/ogus05/ChampSim/tree/master>. UNIST CSE Course Project, accessed June 1, 2025.
- [19] Jee Ho Ryoo, Nagendra Gulur, Shuang Song, and Lizy K. John. 2017. Rethinking TLB Designs in Virtualized Environments: A Very Large Part-of-Memory TLB. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*.
- [20] A. Saulsbury, F. Dahlgren, and P. Stenström. 2000. Recency-based TLB Preloading. In *Proceedings of the 27th Annual International Symposium on Computer Architecture*. IEEE, 117–127.
- [21] Dimitrios Skarlatos, Apostolos Kokolis, Tianyin Xu, and Josep Torrellas. 2020. Elastic Cuckoo Page Tables: Rethinking Virtual Memory Translation for Parallelism. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (Lausanne, Switzerland) (ASPLOS '20)*. Association for Computing Machinery, New York, NY, USA, 1093–1108. doi:10.1145/3373376.3378493
- [22] Yunfang Tai, Wanwei Cai, Qi Liu, Ge Zhang, and Wenzhi Wang. 2013. Comparisons of Memory Virtualization Solutions for Architectures with Software-Managed TLBs. In *Proceedings of the 2013 IEEE 10th International Symposium on Networking, Architecture and Storage (NAS)*.
- [23] Georgios Vavouliotis, Lluc Alvarez, Vasileios Karakostas, Konstantinos Nikas, Nectarios Koziris, Daniel A. Jiménez, and Marc Casas. 2021. Exploiting Page Table Locality for Agile TLB Prefetching. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. 85–98. doi:10.1109/ISCA52012.2021.00016
- [24] Idan Yaniv and Dan Tsafir. 2016. Hash, Don't Cache (the Page Table). In *Proceedings of the 2016 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Science (Antibes Juan-les-Pins, France) (SIGMETRICS '16)*. Association for Computing Machinery, New York, NY, USA, 337–350. doi:10.1145/2896377.2901456